

# [EA&B] TSSMRBench: Benchmarking Memory Systems for Retrieving Correct Temporal Semantic States from Evolving Memories

Hai Lin\*

Shenzhen International Graduate School, Tsinghua  
University  
Shenzhen, China  
Pengcheng Laboratory  
Shenzhen, China  
ngyygm@outlook.com

Hai-Tao Zheng<sup>†</sup>

Shenzhen International Graduate School, Tsinghua  
University  
Shenzhen, China  
Pengcheng Laboratory  
Shenzhen, China  
zheng.haitao@sz.tsinghua.edu.cn

Jianhua Sun\*

Shenzhen International Graduate School, Tsinghua  
University  
Shenzhen, China  
sunjianhua1225@gmail.com

Hui Wang

Pengcheng Laboratory  
Shenzhen, China  
wangh06@pcl.ac.cn

## Abstract

LLM-based agents increasingly rely on external memory systems to maintain information across long-term interactions. However, real-world memories are often not static facts: user preferences, plans, task progress, and event descriptions may evolve through multiple semantic states. Existing benchmarks have advanced the evaluation of long-term memory, temporal reasoning, and knowledge update, but they rarely isolate whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory. To address this gap, we introduce **TSSMRBench**, a benchmark for temporal semantic state memory retrieval.

TSSMRBench formalizes this problem as retrieval over evolving state trajectories and asks whether an agent’s memory system can recover the state evidence required by a query rather than merely a relevant or recent memory. We construct a benchmark with 300 scenarios, 9,000 memory nodes, and 900 multiple choice questions organized into three task families: *single-state lookup*, *cross-version comparison*, and *temporal ordering*. The tasks differ in how many supporting states they require, enabling evaluation of both retrieval precision and support completeness under shared-pool interference.

Experiments with BM25, FAISS, Mem0, and Graphiti show that current retrieval baselines and memory systems perform relatively well on single-state queries but degrade substantially on cross version comparison and temporal ordering queries. Dense retrieval remains the strongest simple baseline, while memory systems expose trade-offs between granularity, support recovery, and latency. These findings show that temporal semantic state retrieval remains challenging for current agent memory systems and suggest that future designs should preserve version history, retrieve coherent state-level evidence, and support stage-aware selection among semantically similar memories.

## CCS Concepts

- **Information systems** → **Information retrieval**; *Data mining*;
- **Computing methodologies** → *Natural language processing*.

## Keywords

agent memory systems, temporal semantic state retrieval, benchmark evaluation, evolving memories

## ACM Reference Format:

Hai Lin, Jianhua Sun, Hai-Tao Zheng, and Hui Wang. 2027. [EA&B] TSSMRBench: Benchmarking Memory Systems for Retrieving Correct Temporal Semantic States from Evolving Memories. In *28th International Conference on Extending Database Technology (EDBT ’27)*, April 6–9, 2027, Lille, France. ACM, New York, NY, USA, 13 pages.

## 1 Introduction

Large language model agents increasingly rely on external memory systems to sustain behavior across long-running interactions. Survey work on LLM-based agent systems describes the rapid expansion of agent architectures and their growing role in complex problem solving [6]. A recent survey focused specifically on memory further argues that memory is what enables an agent to persist, organize, and selectively recall information across interactions [3]. These systems are expected to remember user preferences, track evolving plans, preserve event histories, and support multi-step decision making over time. As agents move from short exchanges to persistent assistance, memory is becoming a first-class systems component rather than a simple extension of context length.

What makes this setting difficult is that many memories evolve. A plan may be revised, a policy may be amended, and an event description may go through several intermediate stages before reaching its latest form. In such settings, the correct answer is often not the most recent memory, but the memory that was valid at a particular stage. Consider a legal assistant asked which revision

\*Hai Lin and Jianhua Sun contributed equally to this work.

<sup>†</sup>Corresponding author.

of a procedural rule first introduced delayed-hearing handling, a project assistant asked to compare the current plan with an earlier draft because an old proposal is being reconsidered, or a financial agent asked to reconstruct the order of several market-state updates before issuing a decision recommendation. These queries require the memory system to recover the right *temporal semantic state*, not merely a topically relevant memory.

This requirement is not well captured by existing evaluation practice. Long-term memory benchmarks such as LoCoMo have substantially improved the evaluation of persistent memory [13]. Temporal and time-sensitive benchmarks such as TimeQA have sharpened our understanding of reasoning under changing information [1]. Retrieval-oriented evaluation has also advanced through RAG-style settings, for which survey work now provides a broad overview of evaluation practices and failure modes [5]. However, these benchmarks usually emphasize retention, relevance, or recency, rather than whether a memory system can distinguish among several semantically similar states of the same evolving object and retrieve the one that matches the queried stage. We discuss these neighboring benchmark families in Section 2. Here, the key point is simple: storing an evolving memory trajectory is not the same as retrieving the stage-correct state from that trajectory.

To study this problem, we introduce **TSSMRBench**, a benchmark for *temporal semantic state memory retrieval* in agent memory systems. TSSMRBench models each scenario as an evolving memory trajectory composed of temporal semantic states and evaluates whether the stage-correct state can be retrieved from a memory system given that trajectory. The benchmark contains three task families: *single-state lookup*, which targets one stage-specific state; *cross-version comparison*, which requires jointly retrieving two stage-specific states for contrast; and *temporal ordering*, which requires recovering several states and arranging them along the correct temporal progression. Figure 1 shows one representative benchmark instance.

The current release of TSSMRBench contains 300 scenarios, 9,000 memory nodes, and 900 multiple choice questions. We evaluate representative sparse retrieval, dense retrieval, and memory-system baselines under a unified global mixed-pool setting, where all memories are stored together before retrieval and each question must be answered from retrieved evidence. The evaluation target is the memory system used by the agent rather than the underlying answer LLM: answer generation is held fixed so that differences primarily reflect memory retrieval quality. Our baseline design spans lexical and dense retrieval together with recent memory-system architectures such as Mem0 [2] and Graphiti [17]. The results show that current memory systems are far more reliable on single-state questions than on comparison and ordering questions; that many failures arise from retrieving temporally incompatible but semantically adjacent states; and that more complex memory systems do not automatically outperform simpler retrievers once state fidelity, memory granularity, and latency are taken into account.

Our contributions are threefold:

- (1) We introduce TSSMRBench and formalize *temporal semantic state memory retrieval*: the ability to retrieve the stage-correct state from a memory system given an evolving memory trajectory.

**Table 1: TSSMRBench at a glance. A scenario is one repository-level release-evolution window, each memory node is one temporal semantic state, and each scenario contributes one question for each task family.**

| Item          | Value                                          |
|---------------|------------------------------------------------|
| Scenarios     | 300 repository-level release windows           |
| Memory nodes  | 9,000 temporal semantic states                 |
| Questions     | 900 multiple choice questions                  |
| Task families | Single-state, cross-version, temporal ordering |

- (2) We build a benchmark dataset with 300 scenarios, 9,000 memory nodes, and 900 questions, and organize it into three task families: single-state lookup, cross-version comparison, and temporal ordering.
- (3) We conduct a systematic experimental study showing that current memory systems still struggle on this problem, especially when semantically similar states coexist, and we use these findings to motivate future designs that preserve version history and support stage-aware retrieval.

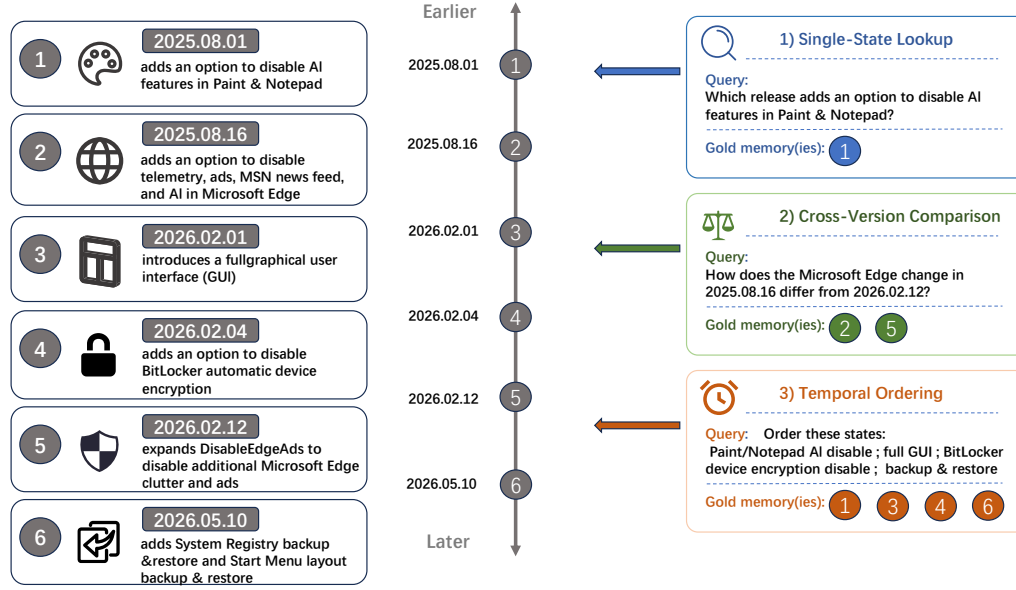
**Outline.** Section 2 reviews related work. Section 3 presents the task definition, design principles, and benchmark construction. Section 4 reports the experimental setup and results. Section 5 concludes. An unnumbered Artifacts section appears immediately before the references.

## 2 Related Work

### 2.1 Agent Memory Systems

Recent work has explored a range of designs for giving LLM agents persistent memory. MemGPT frames long-term memory as an operating-system-style resource-management problem: the LLM decides when to move information between a finite “core memory” (analogous to RAM) and a larger “archival memory” (analogous to disk), enabling persistent multi-turn interaction within a fixed context window [15]. Letta extends this line of work into a broader framework for agent memory operating across sessions and tasks [15]. Mem0 takes a more production-oriented approach: it extracts fine-grained memory facts from user interactions, stores them in a shared vector store, and retrieves the most relevant facts at query time, emphasizing scalability and ease of integration for personalized agents [2]. Graphiti, developed in the Zep ecosystem, organizes memory as a temporal knowledge graph where entities and relationships evolve over time, aiming to preserve richer structural information than flat fact stores [17]. A-MEM introduces a more agentic perspective: the memory system itself decides which memories to create, maintain, or consolidate during ongoing interaction, rather than relying on fixed extraction rules [22].

These systems differ substantially in retrieval granularity (facts, nodes, or graph episodes), memory organization (flat, hierarchical, or graph-structured), and update behavior (append-only, overwrite, or merge). However, they share a common evaluation limitation: they are typically assessed through downstream task success, user-level memory recall, or general question answering. These metrics are valuable, but they do not isolate a narrower question that is central to our benchmark: when several semantically similar states of



**Figure 1: Illustrative benchmark instance from the Raphire/Win11Debloat repository.** The full scenario contains 30 temporal semantic states, but only the task-relevant states are shown here. The single-state lookup question depends on one state, the cross-version comparison question depends on two states, and the temporal ordering question depends on four states. The benchmark therefore tests whether a memory system can retrieve the stage-correct supporting states from an evolving memory trajectory, not merely retrieve a topically related memory.

the same evolving object coexist in memory, can the system retrieve the one that matches the queried stage, rather than the most recent or most topically relevant one? TSSMRBench is designed to test exactly this capability. By constructing explicit versioned state trajectories and asking questions that require stage-specific retrieval, our benchmark provides a targeted diagnostic that complements the broader evaluations used in these systems.

## 2.2 Long-Term Memory Benchmarks

Several recent benchmarks have advanced the evaluation of persistent agent memory. LoCoMo constructs long multi-session conversational scenarios and tests whether a system can recall facts, summaries, and event details mentioned across sessions spanning hundreds of turns [13]. LongMemEval focuses on five memory-related capabilities: knowledge retention, reasoning, forethought, understanding of temporal ordering, and handling of conflicting information, and evaluates them over distributed long histories [21]. MemoryAgentBench examines a complementary setting: agents must reuse information from prior interactions to accomplish later tasks, testing whether memory actually improves downstream performance in a task-oriented pipeline [7].

Taken together, these benchmarks have moved evaluation beyond single-turn QA and made persistent memory a serious empirical target. At the same time, they mainly treat memory as a collection of facts, dialogue details, or episodes to be recalled later. They do not usually build explicit versioned state trajectories for the same underlying object, and they rarely ask for an earlier but

stage-correct state when a later state is also present. In other words, they test whether the system can *remember*, but not whether it can retrieve the *right version* of a memory that has changed over time. TSSMRBench complements this line of work by making this distinction explicit: each scenario contains a full evolution trajectory, and questions are designed so that the correct answer depends on a specific stage, not on the most recent or most similar memory.

## 2.3 Temporal Reasoning and Knowledge Update Benchmarks

A second neighboring family comes from temporal reasoning. Temporal databases and temporal query languages provided early formal tools for representing and querying time-dependent data [8, 18]. In NLP, TORQUE introduces a reading-comprehension dataset focused on event temporal ordering, testing whether models can determine which of two events happened first [14]. TimeDial studies temporal commonsense in dialogue, probing whether models understand implicit temporal relations such as duration and typical event order [16]. TimeQA targets time-sensitive question answering over Wikipedia articles whose facts change over time, asking questions conditioned on a specific time period [1]. ETR-QA provides a comprehensive evaluation of event temporal reasoning abilities, covering multiple reasoning types such as ordering, span estimation, and hypothetical counterfactuals [12]. MusTQ introduces multi-step temporal reasoning over temporal knowledge graphs, requiring models to chain multiple temporal relationships [24]. Test of Time

evaluates LLM temporal reasoning through both semantic understanding and time-arithmetic tasks, measuring whether models can perform explicit date computation [4, 20]. These benchmarks are important because they test whether models can reason about order, duration, and time-conditioned facts. However, they evaluate the *reasoning* capabilities of the model itself rather than the retrieval capabilities of the memory system that feeds the model. In TSSMRBench, the question is not whether the model can order events given their descriptions, but whether the memory system can first retrieve the right version-state evidence from a pool of semantically similar candidates.

A third neighboring family comes from retrieval and evidence-based QA. HotpotQA emphasizes multi-hop evidence aggregation, requiring systems to combine information from multiple retrieved passages [23]. NarrativeQA studies understanding over long narrative evidence, testing comprehension of plot-level events and relationships [9]. Natural Questions evaluates open-domain retrieval and answer extraction from a large web corpus using real user queries [10]. MuSiQue focuses on compositional multi-hop questions that require integrating several pieces of retrieved evidence in a specific order [19]. RAG and its later variants study how retrieved evidence can be incorporated into generation, and survey work now provides a broad overview of retrieval-augmented methods and their failure modes [5, 11]. These works sharpen our understanding of retrieval and evidence use, but they assume a static or unversioned document collection. They do not directly target stage-specific state disambiguation inside an evolving memory trajectory; that is, the retrieval target is a relevant passage, not a specific version of a changing fact.

Finally, knowledge-update settings raise a different but related issue: how a memory system should incorporate newer information and avoid outdated answers. This perspective is highly relevant, but its default assumption is often that newer information ought to dominate older information. TSSMRBench differs in an important way. Our goal is not simply to test whether a memory system prefers the latest state. Instead, we test whether the memory system can retrieve the *correct* state for the queried stage, even when that state is not the most recent one. This distinction matters because in many real-world scenarios (legal rule tracking, policy revision history, project plan evolution) the relevant answer may refer to an intermediate state, not the current one.

### 3 TSSMRBench

Figure 2 summarizes TSSMRBench. This section defines the task (Section 3.1), presents the benchmark evaluation design principles (Section 3.2), describes the data construction flow (Section 3.3) and question design (Section 3.4), and then introduces the evaluation pipeline and metrics (Section 3.5). TSSMRBench focuses specifically on whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory.

#### 3.1 Task Definition

TSSMRBench evaluates *temporal semantic state memory retrieval*: whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory, rather than merely returning a relevant or recent memory.

**Table 2: Comparison of TSSMRBench with representative neighboring benchmarks. LongMem indicates whether the benchmark targets long-term memory; Temporal indicates whether temporal reasoning is a core requirement; Stage-correct retrieval indicates whether the benchmark explicitly asks for the state that is correct for a queried stage rather than merely a relevant or recent memory; Language reports the primary benchmark language; Scale summarizes benchmark size at a coarse level.**

| Benchmark            | Long Mem | Stage-correct |           | Lang. | Scale                     |
|----------------------|----------|---------------|-----------|-------|---------------------------|
|                      |          | Temporal      | retrieval |       |                           |
| LoCoMo [13]          | Yes      | Partial       | No        | EN    | 1,986 QA                  |
| LongMemEval [21]     | Yes      | Partial       | No        | EN    | 500 QA                    |
| MemoryAgentBench [7] | Yes      | Partial       | No        | EN    | Multi-source suite        |
| TimeQA [1]           | No       | Yes           | No        | EN    | ≈40K QA                   |
| MusTQ [24]           | No       | Yes           | No        | EN    | ≈666K QA                  |
| TSSMRBench           | Yes      | Yes           | Yes       | EN    | 300 repositories / 900 QA |

Formally, let

$$\mathcal{T} = (m_1, m_2, \dots, m_n)$$

denote an evolving memory trajectory, where each memory node  $m_i$  records a semantic state of the same underlying object, event, entity, or plan, and where the ordering function

$$\tau(m_i) < \tau(m_j)$$

indicates that  $m_i$  precedes  $m_j$  in the evolution process. A natural-language query  $q$  specifies, either explicitly or implicitly, a stage constraint  $\sigma(q)$  over this trajectory. Let

$$G(q) \subseteq \mathcal{T}$$

be the minimal set of supporting state nodes sufficient to answer  $q$  under  $\sigma(q)$ , and let

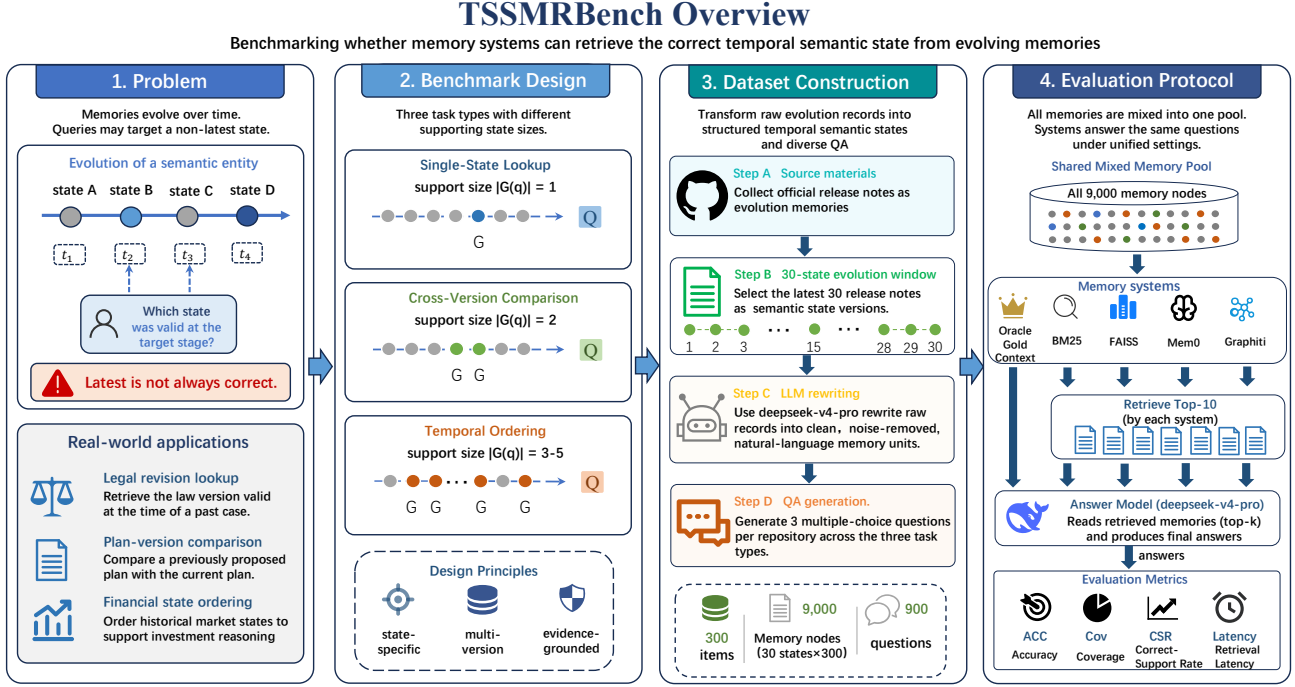
$$R_k(q) = (r_1, r_2, \dots, r_k)$$

be the ranked retrieval output returned by a memory system.

The task is to retrieve a ranked list  $R_k(q)$  such that the supporting set  $G(q)$  is recovered and the answer generator can derive the gold answer  $y(q)$  using only the retrieved evidence. In plain terms, the memory system receives a query about a specific stage of an evolving object and must return exactly those memory nodes that describe the object at that stage. The core difficulty is that several states in  $\mathcal{T}$  may be semantically similar while being temporally incompatible with the query. For example, if a project has gone through ten planning revisions, the memory system must distinguish between the version that first introduced a feature, the version that later modified it, and the current version, even though all ten revisions may describe the same feature in similar language.

TSSMRBench operationalizes this capability through three task families:

- **Single-state lookup:**  $|G(q)| = 1$ . The memory system must identify the one state node that supports the answer.
- **Cross-version comparison:**  $|G(q)| = 2$ . The memory system must recover the two states required to contrast what changed.
- **Temporal ordering:**  $|G(q)| \in \{3, 4, 5\}$ . The memory system must recover enough states to reconstruct a local evolution order.



**Figure 2: Overall framework of TSSMRBench.** The figure summarizes the temporal semantic state retrieval problem, the benchmark task design, the release-derived data construction pipeline, and the unified evaluation protocol. It is intended as a paper-level overview of how the benchmark connects the problem definition, dataset construction, and memory-system evaluation.

**Table 3: Task families and evaluation budgets in TSSMR-Bench.**  $|G(q)|$  denotes the number of supporting states required by a question, and the listed retrieval top-k values are the answer-time cutoffs reused from one shared top-10 retrieval result.

| Task family              | $ G(q) $ | Retrieval top-k |
|--------------------------|----------|-----------------|
| Single-state lookup      | 1        | 1, 2, 3         |
| Cross-version comparison | 2        | 2, 5, 8         |
| Temporal ordering        | 3–5      | 5, 8, 10        |

These tasks differ in support size, but they test the same underlying ability: retrieving the stage-correct state or state set required by the query.

### 3.2 Benchmark Evaluation Design Principles

To isolate this capability, the benchmark and its evaluation protocol follow five principles.

*Stage-specific validity.* Questions must depend on which temporal semantic state is valid for the queried stage, not simply on topical relevance or recency. For instance, a question asking “which version first introduced delayed-hearing handling?” requires the system to identify the earliest matching state, not the latest one.

*Evidence-grounded answering.* The final answer should only be derivable by aligning the question’s state description with retrieved memory content, rather than by exploiting superficial cues such as version labels, dates, or option formatting. This prevents the answer model from guessing the correct option without actually reading the retrieved evidence.

*Varied support requirements.* Some questions depend on one state while others require several states. By varying the support size across task families, the benchmark can distinguish pointwise retrieval (whether the system can find one correct state) from complete support recovery over a trajectory, which requires finding all of the relevant states simultaneously.

*Mixed-memory interference.* All retrieval operates over repository-mixed memory collections rather than isolated per-trajectory stores. This means the memory system cannot exploit trajectory boundaries to narrow the search space, and must instead disambiguate the correct state from a much larger pool of semantically overlapping memories.

*Separating retrieval failure from state-selection failure.* The protocol distinguishes whether a memory system missed the required state entirely (retrieval failure) or instead retrieved semantically related but temporally incompatible evidence (state-selection failure). This distinction is important because the two failure modes

suggest very different system improvements: retrieval failures call for better indexing or recall strategies, while state-selection failures call for better temporal reasoning or version-aware reranking.

### 3.3 Data Construction Flow

We instantiate these principles using an auditable release-only benchmark built from official repository evolution records.

*Step 1: Data source selection.* We use official GitHub release notes because they are repository-authoritative descriptions of evolving software states, including feature additions, policy changes, fixes, migrations, and deprecations. Compared with informal discussions such as commit messages or issue threads, release notes provide a cleaner and more reproducible record of how a system changes across versions. They are also publicly available and version-stamped, which makes the benchmark fully auditable: any result can be traced back to a specific release of a specific repository.

*Step 2: Trajectory node determination.* For each repository, we take the latest 30 usable release notes and treat them as a fixed local evolution window. Each release note becomes one temporal semantic state node, producing an evolving memory trajectory of length 30. We choose 30 nodes because it is long enough to create meaningful interference among semantically similar states (the memory system must pick out the correct stage from dozens of candidates) while still being short enough for the evaluation pipeline to process reliably. Repositories without a sufficient release history are excluded. The formal benchmark contains 300 repositories, yielding 9,000 memory nodes in total.

*Step 3: LLM rewriting of raw release notes.* Raw release notes contain hyperlinks, formatting markers, references, and other artifacts that are not ideal as a memory surface. We therefore use a large language model to rewrite each release note into a normalized English memory unit. This rewriting preserves repository identity and release-specific state content while removing markup-heavy noise. The normalization serves two purposes: it puts all memory units on a level playing field so that retrieval systems are not biased by differences in formatting style across repositories, and it removes information that would make the task trivially easy (e.g., direct links to earlier releases). The resulting normalized text is the only memory surface ingested by the evaluated memory systems; the original release note is retained only for audit and provenance.

*Step 4: QA generation.* On top of each 30-node evolving memory trajectory, we generate one multiple choice question for each task family, yielding 3 questions per repository and 900 questions overall. For every question, we annotate the supporting state nodes required for answering it. This annotation step is essential: without it, we could only measure final answer correctness, but with it we can also measure whether the memory system retrieved the right supporting states regardless of whether the final answer was correct. This makes it possible to evaluate not only final answer correctness but also support recovery.

### 3.4 Question Design

Question design is driven by support complexity rather than by superficial wording diversity.

*Single-state lookup.* These questions ask the memory system to identify one state node that contains a target condition, decision, fix, or release-specific behavior. For example, a question might ask “which release first introduced an option to disable AI features in Paint & Notepad?” The memory system must locate the one state node among 30 candidates that describes this specific behavior, rather than returning any state that merely mentions AI features. They test whether the memory system can localize a single correct state among semantically nearby alternatives.

*Cross-version comparison.* These questions require contrasting two states that differ in a clear, evidence-supported way. For example, a question might ask “what changed between release X and release Y regarding the handling of BitLocker encryption?” Answering requires retrieving both states and identifying the concrete difference between them. We only construct such questions when the source evidence contains a substantive difference between the two releases. Comparison questions are not generated from pairs whose behavior is essentially unchanged.

*Temporal ordering.* These questions require mapping several state descriptions back to their supporting nodes and ordering them correctly along the trajectory. For example, a question might present four feature descriptions and ask which temporal ordering is correct. The difficulty is that the descriptions themselves do not contain explicit dates or version numbers; instead, the answer model must retrieve the corresponding memory nodes and use their content to determine which came first. They are designed so that solving them robustly still depends on retrieving the corresponding memory content, rather than simply relying on superficial ordering cues in the question text alone.

All questions are multiple choice. Version identifiers, tags, or publication cues may appear when they are part of the original material, but the question and options are constrained so that the answer cannot be derived reliably without aligning the state descriptions in the query to the stored memory units.

### 3.5 Evaluation Pipeline and Metrics

All baselines are evaluated under a shared global memory pool: before any query is issued, all 9,000 memory nodes from all 300 repositories are written into each baseline’s own memory backend or retrieval index. Retrieval therefore operates over a single repository-mixed memory collection per baseline, rather than over isolated per-trajectory memory stores. This design choice is intentional: in a real deployed agent, memory from all interactions would coexist in the same store, and the retrieval system would need to disambiguate among overlapping memories without knowing which repository or context they belong to. This setting introduces cross-repository interference and prevents the memory system from exploiting trajectory boundaries.

For each query  $q$ , the evaluated memory system retrieves the top-10 ranked memories:

$$R_{10}(q) = (r_1, \dots, r_{10}).$$

The same ranked list is then reused across task-specific context budgets:

- **single-state lookup:**  $k \in \{1, 2, 3\}$

- **cross-version comparison:**  $k \in \{2, 5, 8\}$
- **temporal ordering:**  $k \in \{5, 8, 10\}$

The answer LLM is allowed to read only these retrieved memories and is not given access to unretrieved memory nodes. This protocol separates retrieval quality from how much of the ranked memory list is exposed at answer time. It also allows us to analyze whether a memory system places the required support near the top of the ranking or only recovers it deeper in the retrieved list.

We report four primary metrics. **Accuracy (ACC)** measures whether the final answer is correct; this is the end-to-end metric that reflects what a user would observe. **Coverage (Cov)** measures how much of the required support set is recovered. In other words, of all the gold supporting states needed to answer the question, what fraction did the memory system actually retrieve:

$$\text{Cov}(q, k) = \frac{|G(q) \cap R_k(q)|}{|G(q)|}.$$

**Complete Support Rate (CSR)** measures whether the full support set is recovered: a binary indicator that is 1 only when every required supporting state appears in the retrieved list.

$$\text{CSR}(q, k) = 1[G(q) \subseteq R_k(q)].$$

**Latency** measures retrieval time only, excluding answer generation.

Because TSSMRBench is intended to diagnose memory retrieval rather than only final QA success, these metrics are designed to jointly capture answer correctness, support completeness, and retrieval cost.

## 4 Experiments

### 4.1 Experimental Setup

The formal benchmark is built from 300 repositories, yielding 300 repository-level release-evolution scenarios, 9,000 memory nodes, and 900 multiple choice questions. We compare five baselines under the same benchmark inputs: BM25, FAISS, Mem0, Graphiti, and Oracle Gold Context.

BM25 indexes the normalized memory units directly and serves as a sparse lexical retriever. FAISS indexes dense node embeddings and serves as a standard semantic retriever. Mem0 extracts fine-grained memory facts from the ingested text and retrieves over those facts; at query time, it returns the top-10 fact groups, and for node-level coverage metrics, these fact groups are aligned back to the source state-version nodes using the grouped source-node identifiers provided by Mem0. Graphiti builds a temporal knowledge graph over the ingested memories and applies a cross-encoder reranker to the retrieved candidates. Oracle Gold Context does not perform retrieval; instead, it directly provides the answer model with the annotated supporting memory units required by each question. We include Oracle Gold Context as an ideal-retrieval upper bound: it measures how much accuracy headroom remains once the correct state evidence is fully available to the answer model.

All baselines use the same normalized memory units and the same answer-generation model. In the final evaluation, answer generation and LLM-based judging both use deepseek-v4-pro with temperature 0.0 and thinking disabled. Dense retrieval uses BGE-M3 embeddings. Mem0 and Graphiti use deepseek-v4-flash as

their internal memory-processing model, and Graphiti additionally uses a BGE Reranker v2 M3 model. The answer model is therefore controlled across baselines, so the benchmark targets differences in memory retrieval rather than differences in the underlying LLM.

### 4.2 Overall Results

Table 5 reports the main results. Before discussing the individual systems, it is helpful to read the table along two dimensions: across rows (how each system performs as the task gets harder) and across columns within each task block (how performance changes as the retrieval budget  $k$  increases). Oracle Gold Context is treated as ideal retrieval, so its Cov and CSR are 1.0 by construction because the annotated supporting states are directly provided to the answer model. Its remaining errors appear only in ACC. The gap between Oracle and the automatic systems therefore reflects how much accuracy is lost due to imperfect retrieval.

Among the automatic systems, FAISS is the strongest overall baseline. It achieves the best non-oracle ACC, Cov, and CSR on most task and retrieval-top-k settings, and it is the only automatic system that remains relatively stable as the task moves from single-state lookup to cross-version comparison and temporal ordering. This makes dense node-level retrieval a strong reference point for the benchmark. However, FAISS still degrades substantially on the harder tasks, which shows that semantic similarity alone is not sufficient once multiple versions are topically close but temporally different. One notable exception is cross-version comparison at  $k=2$ , where Graphiti (ACC 0.5467) slightly outperforms FAISS (ACC 0.5033), suggesting that graph-based structure and reranking can help when the retrieval budget is tight.

The other baselines reveal complementary weaknesses. BM25 remains serviceable on single-state lookup but degrades sharply on multi-state tasks, especially when the correct versions share substantial vocabulary with nearby distractor versions. Mem0 lags behind FAISS and Graphiti on all three task families, with the largest deficits on cross-version comparison and temporal ordering. This pattern suggests a granularity mismatch: retrieving fine-grained memory facts is not the same as recovering complete version states, because the benchmark’s gold support is defined over coherent version-level nodes rather than over isolated facts. Graphiti is stronger than BM25 and Mem0 on the harder tasks and is competitive with FAISS on cross-version comparison at low  $k$ , but it still trails FAISS overall, suggesting that structured memory organization and reranking do not by themselves solve version disambiguation at larger retrieval budgets.

### 4.3 Results by Task Type and Top-k

Figure 3 reports the results by task family and retrieval top-k.

The first trend concerns the effect of larger retrieval top-k. Coverage usually increases with  $k$ , especially on cross-version comparison and temporal ordering, but answer accuracy does not rise at the same rate. This gap is clearest for Mem0 and Graphiti. On temporal ordering, Mem0’s COV rises from 0.5556 at  $k=5$  to 0.6356 at  $k=10$ , while ACC rises only from 0.1867 to 0.2600. Graphiti shows a similar pattern: its temporal ordering COV increases from 0.6503 to 0.7136, yet ACC slightly decreases from 0.4833 at  $k=8$  to 0.4767 at  $k=10$ . Even Oracle Gold Context shows a mild version of this

**Table 4: Systems and configurations used in the formal benchmark. Oracle Gold Context bypasses retrieval and directly supplies the annotated supporting memory units to the answer model.**

| System              | Retrieval unit                    | Internal memory model | Embedding / reranking                  | Answer model    |
|---------------------|-----------------------------------|-----------------------|----------------------------------------|-----------------|
| BM25                | memory unit text                  | none                  | lexical matching                       | DeepSeek-V4-Pro |
| FAISS               | memory unit text                  | none                  | BGE-M3 embeddings                      | DeepSeek-V4-Pro |
| Mem0                | memory facts                      | DeepSeek-V4-Flash     | BGE-M3 embeddings                      | DeepSeek-V4-Pro |
| Graphiti            | graph episodes / nodes            | DeepSeek-V4-Flash     | BGE-M3 embeddings + BGE Reranker v2 M3 | DeepSeek-V4-Pro |
| Oracle Gold Context | annotated supporting memory units | none                  | none                                   | DeepSeek-V4-Pro |

**Table 5: Main results on TSSMRBench. Each cell reports ACC / Cov / CSR, where ACC is answer accuracy, Cov is the average fraction of gold supporting states recovered in the retrieved set, and CSR is the fraction of questions for which the full gold support set is recovered. Each task family is evaluated at its own retrieval top- $k$  settings: single-state lookup uses  $k=1, 2, 3$ ; cross-version comparison uses  $k=2, 5, 8$ ; temporal ordering uses  $k=5, 8, 10$ . Oracle Gold Context is shown as an upper bound and is excluded from strongest-baseline marking. Bold marks the best automatic baseline and underline marks the second-best automatic baseline for each metric under the same task and top- $k$  setting.**

| Task                     | System              | Retrieval top- $k$              |                                 |                                 |
|--------------------------|---------------------|---------------------------------|---------------------------------|---------------------------------|
|                          |                     | $k=1$                           | $k=2$                           | $k=3$                           |
| Single-state lookup      | BM25                | 0.8533 / 0.8333 / 0.8333        | 0.8633 / 0.8733 / 0.8733        | 0.8800 / 0.8900 / 0.8900        |
|                          | FAISS               | <b>0.8933 / 0.8667 / 0.8667</b> | <b>0.9133 / 0.9200 / 0.9200</b> | <b>0.9433 / 0.9500 / 0.9500</b> |
|                          | Mem0                | 0.6433 / 0.6733 / 0.6733        | 0.6533 / 0.7033 / 0.7033        | 0.6567 / 0.7100 / 0.7100        |
|                          | Graphiti            | 0.6800 / 0.6467 / 0.6467        | 0.7300 / 0.7167 / 0.7167        | 0.7267 / 0.7267 / 0.7267        |
|                          | Oracle Gold Context | 0.9967 / 1.0000 / 1.0000        | 1.0000 / 1.0000 / 1.0000        | 1.0000 / 1.0000 / 1.0000        |
| Cross-version comparison |                     | $k=2$                           | $k=5$                           | $k=8$                           |
|                          | BM25                | 0.2100 / 0.2767 / 0.1233        | 0.3200 / 0.3933 / 0.2567        | 0.3933 / 0.4483 / 0.3033        |
|                          | FAISS               | 0.5033 / 0.6033 / 0.3433        | <b>0.7467 / 0.8067 / 0.6467</b> | <b>0.8400 / 0.8917 / 0.7933</b> |
|                          | Mem0                | 0.2700 / 0.4400 / 0.1900        | 0.4067 / 0.5917 / 0.3833        | 0.4467 / 0.6400 / 0.4567        |
|                          | Graphiti            | <b>0.5467 / 0.6200 / 0.3933</b> | 0.6567 / 0.7150 / 0.5100        | 0.6700 / 0.7417 / 0.5467        |
|                          | Oracle Gold Context | 0.9900 / 1.0000 / 1.0000        | 0.9833 / 1.0000 / 1.0000        | 0.9800 / 1.0000 / 1.0000        |
| Temporal ordering        |                     | $k=5$                           | $k=8$                           | $k=10$                          |
|                          | BM25                | 0.2767 / 0.5406 / 0.2033        | 0.3267 / 0.5803 / 0.2600        | 0.3200 / 0.6094 / 0.2867        |
|                          | FAISS               | <b>0.4633 / 0.7036 / 0.3033</b> | <b>0.5633 / 0.8200 / 0.5200</b> | <b>0.6300 / 0.8606 / 0.6233</b> |
|                          | Mem0                | 0.1867 / 0.5556 / 0.1733        | 0.2500 / 0.6136 / 0.2733        | 0.2600 / 0.6356 / 0.3067        |
|                          | Graphiti            | 0.3767 / 0.6503 / 0.2733        | 0.4833 / 0.6975 / 0.3533        | 0.4767 / 0.7136 / 0.3767        |
|                          | Oracle Gold Context | 0.9067 / 1.0000 / 1.0000        | 0.9100 / 1.0000 / 1.0000        | 0.9100 / 1.0000 / 1.0000        |

effect on cross-version comparison. Larger retrieval top- $k$  therefore helps recover more gold evidence, but it can also introduce more temporally adjacent distractors and increase the burden of evidence integration.

The second trend is a consistent difficulty ordering across baselines: single-state lookup is easiest, cross-version comparison is harder, and temporal ordering is hardest. As the number of required supporting states grows from one to two and then to three-to-five, the evaluated memory systems and retrieval baselines must do more than retrieve a relevant topic; they must identify a temporally compatible subset of a version trajectory. This pattern also helps explain the relative behavior of the baselines. FAISS remains strongest because dense node retrieval is effective at locating semantically close versions, but its performance still drops once several nearby versions are all topically relevant. BM25 declines more sharply because lexical overlap is an even coarser proxy for version identity. Mem0 gains coverage as top- $k$  grows, yet much of the added evidence arrives as fragmented facts rather than coherent state versions, which weakens the conversion from retrieval support to

correct answers. Graphiti benefits from structure and reranking on the harder tasks, but its gains still plateau, suggesting that retrieving the right memory region is not the same as preserving the fine-grained version boundaries required by these questions.

To quantify the task effect beyond the descriptive plots, we fit a mixed-effects logistic regression on answer correctness using the automatic baselines at the main task-specific retrieval top- $k$  settings. This statistical model allows us to isolate the effect of task difficulty from the effect of system choice and from question-level variation. The model uses fixed effects for task family and system, with a random intercept for question. Table 6 shows that, relative to single-state lookup, the odds of a correct answer drop to 0.26 $\times$  for cross-version comparison and to 0.11 $\times$  for temporal ordering. This provides model-based support for the same conclusion suggested by the curves: retrieving the stage-correct state becomes substantially harder once the question depends on multiple temporally related versions.



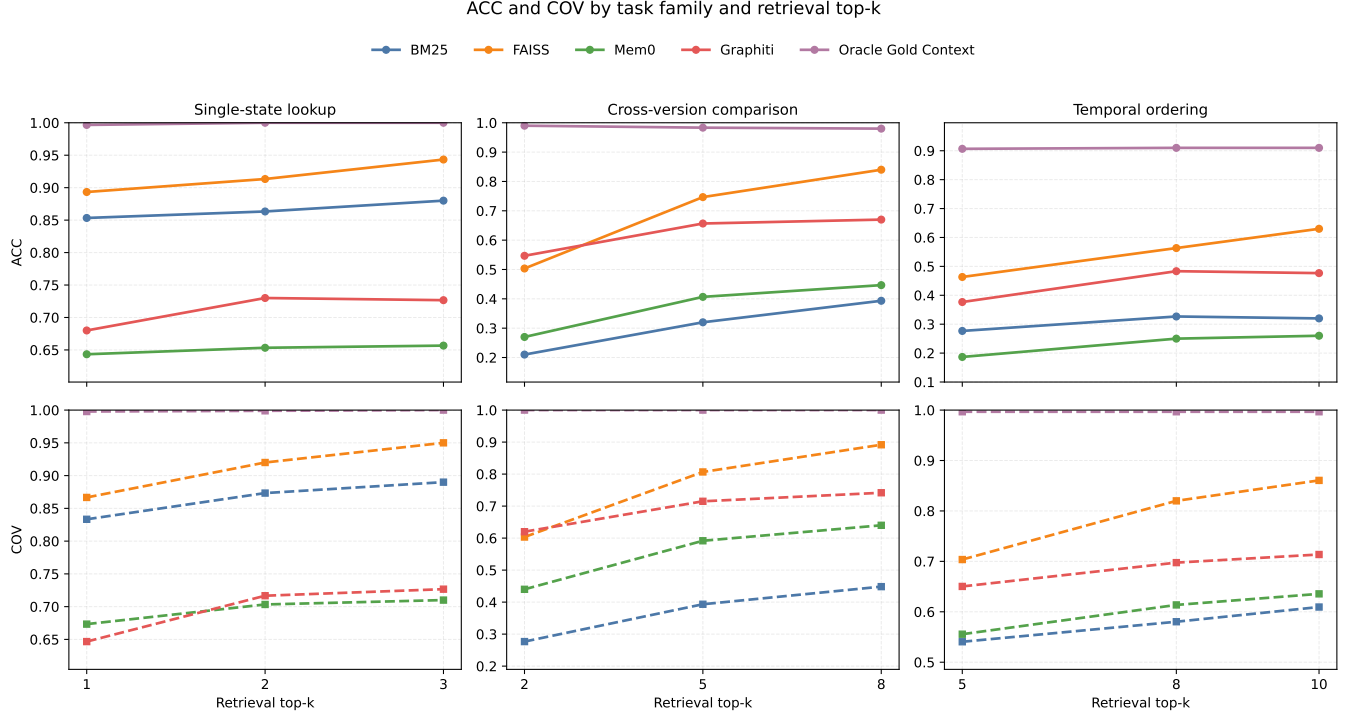


Figure 3: ACC and Cov by task family and retrieval top-k. The top row reports ACC and the bottom row reports Cov; the three columns correspond to single-state lookup, cross-version comparison, and temporal ordering.

Table 6: Mixed-effects logistic regression on answer correctness at the main task-specific retrieval top-k settings. The model uses fixed effects for task family and system, with a random intercept for question. Odds ratios are reported relative to single-state lookup.

| Contrast           | Odds ratio | 95% interval |
|--------------------|------------|--------------|
| Cross vs Single    | 0.26       | [0.23, 0.30] |
| Temporal vs Single | 0.11       | [0.09, 0.12] |

#### 4.4 Decoupling Retrieval from Reasoning

To better diagnose failure modes, we decouple retrieval success from final answer correctness. For each question, we categorize the retrieval outcome into three buckets: *all-recalled*, where all annotated supporting states are retrieved; *partial-recalled*, where only part of the support set is retrieved; and *zero-recalled*, where none of the supporting states are retrieved. We then split each bucket by whether the final answer is correct or incorrect.

This analysis is important because overall answer accuracy alone cannot tell us whether a memory system fails by missing relevant memories entirely, by retrieving incomplete support, or by retrieving related but temporally incompatible evidence. In particular, the *all-recalled but incorrect* bucket is highly diagnostic: it indicates that the memory system has already retrieved the necessary state evidence but the final answer is still not resolved correctly. Conversely, the *partial-recalled but correct* and *zero-recalled but correct* buckets

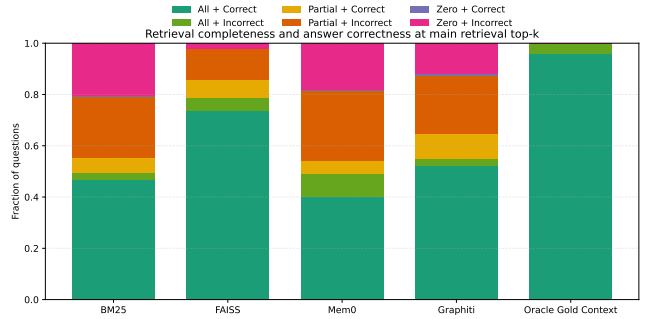


Figure 4: Retrieval–reasoning decoupling at the main task-specific retrieval top-k settings. Each bar partitions questions by retrieval completeness and final answer correctness.

indicate cases where the answer may be guessed from partial cues or option bias rather than supported by complete evidence.

Figure 4 shows that the baselines fail for different reasons, and these differences are informative from a systems perspective. FAISS has the strongest retrieval profile: 73.44% of all questions fall into the all-recalled and correct bucket, while only 2.00% fall into zero-recalled and incorrect. Its remaining errors are therefore less about complete retrieval failure and more about selecting among semantically adjacent versions after the right region of memory has already

**Table 7: Computational cost profile. Latency is mean retrieval-only latency in milliseconds, excluding answer generation. Retrieved context length is the mean number of context tokens passed to the answer model at the primary retrieval top-k setting of each task family.**

| System              | Latency  | Single | Cross  | Temporal |
|---------------------|----------|--------|--------|----------|
| BM25                | 1110.63  | 181.41 | 402.16 | 438.14   |
| FAISS               | 195.12   | 270.17 | 611.98 | 856.29   |
| Mem0                | 417.71   | 90.72  | 174.46 | 214.38   |
| Graphiti            | 34425.76 | 224.58 | 539.11 | 836.39   |
| Oracle Gold Context | 0.00     | 87.64  | 169.43 | 309.78   |

been retrieved. This is consistent with the behavior of dense semantic search: it is good at getting close to the target state, but it has no explicit mechanism for preserving version lineage or distinguishing neighboring historical states.

Mem0 presents a different pattern once its fact-level retrieval is aligned back to source state nodes through grouped source-node identifiers. Under this corrected accounting, only 0.22% of all questions fall into zero-recalled and correct, confirming that Mem0 rarely answers correctly without relevant evidence. Its dominant error buckets are instead partial-recalled and incorrect (27.44%) and all-recalled but incorrect (8.89%). Because Mem0 retrieves individual memory facts rather than complete version states, it frequently recovers only part of the required support. In multi-state questions, this partial recovery weakens support completeness and makes it harder for the answer model to reconstruct a temporally compatible trajectory.

Graphiti suggests a third failure mode. It is stronger than Mem0 on cross-version comparison and temporal ordering, which shows the benefit of structure and reranking, but it still has a large partial-recalled and incorrect segment (22.78%). Graphiti’s graph construction process builds temporal edges and summarized entity nodes over the ingested memories, which can help organize long histories but may also reduce the distinctiveness of individual version states. For TSSMRBench, this is harmful because the correct answer often depends on preserving distinct historical states rather than collapsing them into one generalized representation.

These findings point to a broader design implication. A memory system for evolving information cannot rely only on semantic relevance, nor can it simply overwrite old memories with newer ones. It must preserve version-level history, retrieve coherent state units rather than only fragments, and support multi-state evidence assembly without losing temporal compatibility.

#### 4.5 Computational Efficiency

The previous sections focused on retrieval quality. However, for any memory system that must operate in real time, latency and cost are equally important. This section examines the computational efficiency of each baseline.

Table 7 shows the results. FAISS is the strongest efficiency–effectiveness trade-off among the automatic baselines: it is the fastest retriever (195.12 ms on average) and also the strongest automatic system in accuracy, coverage, and complete-support rate. BM25 is slower than FAISS despite weaker overall effectiveness,

which makes it a useful lexical reference point but not a strong efficiency frontier.

Mem0 occupies a different point in the trade-off space. Its latency remains moderate, and its retrieved contexts are the shortest among the automatic systems because it operates on compact memory facts. However, this apparent efficiency advantage comes with a clear loss in state-level effectiveness. The system retrieves smaller memory units quickly, but those units are often too fragmented to support reliable version-level reasoning. In other words, cheaper context is not necessarily better context.

Graphiti exposes the opposite extreme. It retrieves contexts of similar scale to FAISS on the harder tasks, but its retrieval latency is much higher (34,425.76 ms on average). This cost likely reflects the additional graph traversal, candidate expansion, and reranking needed to retrieve structured memories from a global mixed pool. For TSSMRBench, the practical question is whether this extra structure leads to clearly better version-sensitive retrieval at an acceptable cost. Our results show that this is not yet guaranteed. Future memory systems therefore need to satisfy both requirements at once: they should preserve version history faithfully, and they should still retrieve the needed states efficiently from large mixed memory pools.

## 5 Conclusion

We presented TSSMRBench, a benchmark for temporal semantic state memory retrieval over evolving memories. TSSMRBench evaluates whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory, rather than merely returning a relevant or recent memory.

To study this ability, we constructed a benchmark dataset with 300 scenarios, 9,000 memory nodes, and 900 questions, organized into three task families: single-state lookup, cross-version comparison, and temporal ordering. This design lets us test not only whether a memory system retrieves relevant memory, but also whether it retrieves the temporally correct state evidence needed by questions of increasing multi-state complexity.

Our experiments show that current memory systems are substantially stronger on single-state lookup than on cross-version comparison and temporal ordering, and that many failures arise even when semantically related evidence is present in memory. Sparse and dense retrieval backends often retrieve topically relevant versions but do not explicitly preserve version identity; fact-based memory systems recover individual facts rather than complete version states, leading to partial support; and structured memory systems can reduce the distinctiveness of individual historical states through graph construction and summarization. These results highlight the challenge of temporal semantic state retrieval for current memory systems and clarify an important future design direction: memory systems should preserve version history, retrieve coherent state-level evidence, support stage-aware selection among semantically similar memories, and do so efficiently over large shared memory pools.

Looking ahead, several extensions are natural. The current benchmark is English-only and derived from software release notes; applying the same evaluation methodology to other domains (such as legal document revision, medical guideline updates, or policy

amendment tracking) would test whether the observed failure patterns generalize across content types and languages. On the systems side, the gap between dense retrieval and dedicated memory architectures suggests that a lightweight version-aware reranking layer, placed on top of any existing retriever, might be a practical and high-impact improvement. We hope TSSMRBench provides a concrete evaluation foundation for this line of work.

## Artifacts

**Repository.** The complete source code, benchmark dataset, evaluation pipeline, and raw result files are publicly available at: <https://github.com/ngyygm/llm-TSSMRBench>

**Contents.** The repository includes: (1) the formal benchmark dataset (official\_300\_merged.json) containing 300 repository-level release-evolution windows with 9,000 memory nodes and 900 annotated multiple choice questions; (2) evaluation scripts that reproduce all reported experiments for BM25, FAISS, Mem0, and Graphiti baselines; (3) per-system result files (\*.questions.jsonl) retaining the full prompt payload, retrieved-memory text, matched supporting node identifiers, and per- $k$  sliced evaluation outputs so that every reported score can be traced to concrete evidence; and (4) configuration files for all baseline systems.

**Usage.** To reproduce the evaluation results: (1) install dependencies as specified in the repository README; (2) run the evaluation pipeline with the provided configuration files; (3) use the included analysis scripts to regenerate all tables and figures reported in this paper. The benchmark dataset is self-contained and can also be loaded directly for custom evaluation.

**Dataset mirror.** The benchmark dataset is additionally available on HuggingFace Datasets for convenient access.

**License.** The benchmark dataset and evaluation code are released under the MIT License.

## A Appendix

This appendix provides details supporting the reproducibility and interpretability of TSSMRBench. Section A.1 presents the prompts used during benchmark construction and answer generation. Section A.2 explains the dataset schema. Section A.3 illustrates retrieval success and failure patterns with concrete examples.

### A.1 Construction and Evaluation Prompts

TSSMRBench uses LLM-assisted construction at two stages: (1) rewriting raw release notes into normalized memory units, and (2) generating multiple choice questions with annotated supporting states. We present the key excerpts and explain the design choices behind them.

*Memory-unit construction.* Listing 1 shows the core instructions. Several constraints are critical for benchmark integrity. First, each normalized memory unit must describe *only the current release note* without comparing it to earlier or later releases; this prevents cross-version leakage that would trivialize retrieval. Second, each unit must explicitly name both the repository and the release version or tag, ensuring that version identity is preserved. Third, the prompt forbids mentioning benchmark setup, memory pools, or neighboring releases, so that the resulting text resembles a natural memory entry rather than an artificially constructed artifact.

#### Listing 1: Excerpt from the release-window construction prompt.

```
Build one unified repository release-evolution prototype from the provided
official GitHub release-note nodes.

Requirements:
- Use only the provided release-note nodes.
- Produce one single JSON object matching the output schema.
- Only generate chunk-level normalized text values keyed by node identifier.
- For every chunk, write in English.
- Each text must explicitly name the repository and the current release version
  or tag.
- Each text must describe only the current release note itself.
- Do not compare to earlier or later releases.
- Do not mention benchmark setup, memory pools, retrieval, or neighboring
  releases.
...
```

*QA generation schema.* Listing 2 shows the output schema and rules. Each repository receives exactly three questions, one per task family. Every question must include the gold supporting node identifiers, which make it possible to compute coverage and complete-support rate independently of final answer correctness. The rules enforce task-specific constraints: cross-version comparison questions must be grounded in concrete differences (not trivially unchanged pairs), and temporal-ordering questions must use 3–5 states whose ordering requires reading the actual memory content rather than relying on version-number patterns.

#### Listing 2: Excerpt defining the output schema and QA rules.

```
"questions": [
{
  "question_id": "string",
  "task_type": "single_state_lookup / cross_version_comparison /
    temporal_version_ordering",
  "answer_format": "multiple_choice",
  "query_text": "question text that explicitly names the repository",
  "options": [
    { "option_id": "A", "text": "option text", ...
  ],
  "correct_option_id": "A",
  "source_chunk_ids": ["node_id_1"],
  "answer_support": [{"node_id": "node_id_1", "answer_text": "support text"}]
}
]

- Generate exactly 3 questions, one per task type.
- Every query must explicitly name the repository.
- For cross_version_comparison, compare two releases using concrete differences.
- For temporal_version_ordering, use 3 to 5 release-content states.
- The answer should require retrieving the corresponding memory content.
...
```

*Answer-generation prompt.* Listing 3 shows the answer-generation prompt applied identically across all baselines. Two design choices are worth noting. First, the prompt instructs the model to answer *only from the retrieved memory*, explicitly forbidding outside knowledge or guessing, so that the final answer reflects what the memory system provided. Second, the model may return option E when the retrieved memory is completely irrelevant, which prevents forced guessing and makes accuracy a more faithful measure of retrieval quality.

### A.2 Dataset Schema

The benchmark dataset stores each scenario as a JSON object containing the memory trajectory and the associated questions. We explain the two core record types.

*Memory-node record.* Each memory node contains a unique identifier encoding the repository, source type, and date-derived key.

The raw release-note text is preserved for auditing, while the normalized text (produced by the LLM rewriting step) is the only text written into the retrieval pool. The normalization removes hyperlinks, formatting markers, and cross-release comparisons, producing a clean description of the release-specific state. Timestamp fields provide the temporal grounding for the node’s position in the evolution trajectory. Separating the raw and normalized versions ensures that the benchmark measures retrieval over a controlled memory surface.

### Listing 3: Excerpt from the answer-generation prompt.

```
You are a strict evaluation assistant for a memory benchmark.

You must answer using only the retrieved memory.
You must not use outside knowledge, background knowledge, guessing,
or prior world knowledge.
Your job is to choose the best-supported answer from the retrieved memory.

Output rule:
- Return exactly one uppercase option letter: A, B, C, D, or E.
- Do not output any explanation, punctuation, JSON, markdown, or extra text.
- If the retrieved memory contains relevant evidence for one option,
  choose that option actively.
- Return E only when the retrieved memory is completely irrelevant
  or contains no answer-bearing evidence for any option.
...
```

*Question record.* Each question record includes the task type, the query text, four multiple choice options, the correct option, and the annotated gold supporting nodes. The gold node identifiers make it possible to evaluate not only whether the final answer is correct but also whether the memory system retrieved the right supporting states. For temporal-ordering questions, the options are orderings of state indices (e.g.,  $2 \rightarrow 1 \rightarrow 4 \rightarrow 3$ ), so the question cannot be solved by pattern-matching version numbers alone. Each question also includes an `answer_support` field that provides per-node evidence text explaining why each supporting node is relevant, useful for auditing gold annotations.

### A.3 Retrieval Result Examples

To illustrate retrieval-level behavior at the individual question level (complementing the aggregate analysis in Section 4.4), we describe one successful and one failed case from the Mem0 evaluation.

*Successful single-state retrieval.* In this case, Mem0 retrieves three memory facts, all topically related to the query. The first retrieved fact matches the gold node exactly, yielding a support coverage of 1.0, and the answer model correctly selects the right option. This illustrates a case where Mem0’s fact-level retrieval aligns well with the benchmark’s state-level gold support: the relevant fact is present, and no competing version facts introduce confusion. The second and third retrieved facts come from nearby releases that mention related but different features; this is exactly the kind of semantically adjacent distractor that makes the task non-trivial, yet the correct fact still ranks first.

*Failed temporal-ordering retrieval.* In this case, the gold support requires four state nodes, but Mem0 retrieves only one matching fact, yielding a support coverage of 0.25. The remaining retrieved facts are topically related (e.g., UI improvements, feature tweaks) but do not correspond to the other three gold nodes. Because the answer model cannot reconstruct the full temporal ordering from partial evidence, it returns E (no answer-bearing evidence), and the

**Table 8: Per-task retrieval–reasoning decoupling at the main task-specific top- $k$  settings. Each cell reports the fraction of questions in each retrieval completeness bucket. All-recalled: full support recovered; partial-recalled: only part of the support recovered; zero-recalled: no supporting states recovered.**

| Task     | System   | All  | Partial | Zero |
|----------|----------|------|---------|------|
| Single   | BM25     | 0.89 | 0.00    | 0.11 |
|          | FAISS    | 0.95 | 0.00    | 0.05 |
|          | Mem0     | 0.71 | 0.00    | 0.29 |
|          | Graphiti | 0.73 | 0.00    | 0.27 |
| Cross    | BM25     | 0.30 | 0.29    | 0.41 |
|          | FAISS    | 0.79 | 0.20    | 0.01 |
|          | Mem0     | 0.46 | 0.37    | 0.18 |
|          | Graphiti | 0.55 | 0.39    | 0.06 |
| Temporal | BM25     | 0.29 | 0.61    | 0.11 |
|          | FAISS    | 0.62 | 0.37    | 0.00 |
|          | Mem0     | 0.31 | 0.61    | 0.09 |
|          | Graphiti | 0.38 | 0.59    | 0.04 |

question is scored as incorrect. This case illustrates the granularity mismatch discussed in Section 4.4: retrieving individual facts rather than complete version states is often insufficient when a question depends on multiple temporally ordered states. More broadly, this failure mode suggests that fact-oriented memory systems may need an additional mechanism (such as explicit version grouping or trajectory-aware retrieval) to assemble multi-state evidence as a coherent unit rather than as a bag of independently retrieved facts.

*Data provenance.* In the formal experiments, the evaluated memory systems ingest only the normalized text. Raw release-note text is preserved in the dataset but is never written into the retrieval pool. The released result files also retain the original prompt payload, the full retrieved-memory text, the matched supporting node identifiers, and per- $k$  sliced evaluation outputs, so that every reported score can be traced back to a concrete record without re-running the full pipeline. This provenance is important because a benchmark that aims to diagnose system behavior, not only rank systems, must allow reviewers to inspect the evidence behind any individual result.

### A.4 Per-Task Retrieval–Reasoning Decoupling

Table 8 breaks down the retrieval–reasoning decoupling from Section 4.4 by task family. On single-state lookup, the partial-recalled bucket is empty by construction because the gold support consists of exactly one node. On cross-version comparison and temporal ordering, partial-recalled becomes the largest bucket for most systems, reflecting the increasing difficulty of recovering all required states simultaneously. FAISS achieves the highest all-recalled rate across all three tasks, but still drops from 0.95 on single-state lookup to 0.62 on temporal ordering, showing that even dense retrieval struggles to recover the full support set when multiple temporally related states are needed. A notable contrast is between Mem0 and Graphiti on temporal ordering: Mem0 has a higher zero-recalled rate (0.09 vs. 0.04) but a similar partial-recalled rate, suggesting that Graphiti’s graph traversal more reliably reaches the right memory region but still loses individual version boundaries during graph construction.

## References

- [1] Wenhui Chen, Xinyi Wang, and William Yang Wang. 2021. A Dataset for Answering Time-Sensitive Questions. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Curran Associates, Inc., Virtual. <https://arxiv.org/abs/2108.06314>
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413 [cs.CL]* doi:10.48550/arXiv.2504.19413
- [3] Pengfei Du. 2026. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. *arXiv:2603.07670 [cs.AI]* doi:10.48550/arXiv.2603.07670
- [4] Bahare Fatemi, Mehran Kazemi, Anton Tsitsulin, Karishma Malkan, Jinyeong Yim, John Palowitch, Sungyong Seo, Jonathan Halcrow, and Bryan Perozzi. 2024. Test of Time: A Benchmark for Evaluating LLMs on Temporal Reasoning. *arXiv:2406.09170 [cs.CL]* <https://arxiv.org/abs/2406.09170>
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haoften Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997 [cs.CL]* doi:10.48550/arXiv.2312.10997
- [6] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv:2402.01680 [cs.CL]* doi:10.48550/arXiv.2402.01680
- [7] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. *arXiv:2507.05257 [cs.CL]* doi:10.48550/arXiv.2507.05257
- [8] Christian S. Jensen and Richard T. Snodgrass. 1999. Temporal Data Management. *IEEE Transactions on Knowledge and Data Engineering* 11, 1 (1999), 36–44. doi:10.1109/69.755613
- [9] Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, and Gábor Melis. 2018. The NarrativeQA Reading Comprehension Challenge. *Transactions of the Association for Computational Linguistics* 6 (2018), 317–328. doi:10.1162/tacl\_a\_00023
- [10] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl\_a\_00276
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Virtual, 9459–9474.
- [12] Sigang Luo, Yanan Liu, Dongying Lin, Yingying Zhai, Bin Wang, Xiaochun Yang, and Junpeng Liu. 2025. ETRQA: A Comprehensive Benchmark for Evaluating Event Temporal Reasoning Abilities of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 23321–23339. doi:10.18653/v1/2025.findings-acl.1198
- [13] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 13851–13870. doi:10.18653/v1/2024.acl-long.747
- [14] Qiang Ning, Hao Wu, Rujun Han, Nanyun Peng, Matt Gardner, and Dan Roth. 2020. TORQUE: A Reading Comprehension Dataset of Temporal Ordering Questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online, 1158–1172. doi:10.18653/v1/2020.emnlp-main.88
- [15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560 [cs.AI]* doi:10.48550/arXiv.2310.08560
- [16] Lianhui Qin, Aditya Gupta, Shyam Upadhyay, Luheng He, Yejin Choi, and Manaal Faruqui. 2021. TIMEDIAL: Temporal Commonsense Reasoning in Dialog. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 7066–7076. doi:10.18653/v1/2021.acl-long.549
- [17] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956 [cs.CL]* doi:10.48550/arXiv.2501.13956
- [18] Richard T. Snodgrass. 1995. *The SQL2 Temporal Query Language*. Kluwer Academic Publishers, Boston, MA.
- [19] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554. doi:10.1162/tacl\_a\_00475
- [20] Shaohang Wei, Wei Li, Feifan Song, Wen Luo, Tianyi Zhuang, Haochen Tan, Zhijiang Guo, and Houfeng Wang. 2025. TimeE: A Multi-level Benchmark for Temporal Reasoning of LLMs in Real-World Scenarios. *arXiv:2505.12891 [cs.CL]* doi:10.48550/arXiv.2505.12891
- [21] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv:2410.10813 [cs.CL]* doi:10.48550/arXiv.2410.10813
- [22] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv:2502.12110 [cs.AI]* doi:10.48550/arXiv.2502.12110
- [23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [24] Tingyi Zhang, Jiaan Wang, Zhixu Li, Jianfeng Qu, An Liu, Zhigang Chen, and Hongping Zhi. 2024. MusTQ: A Temporal Knowledge Graph Question Answering Dataset for Multi-Step Temporal Reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 11688–11699. doi:10.18653/v1/2024.findings-acl.696